Laboratory 2

Variant 4

Group 24

By Gülten Sezer and Sai Koushik Neriyanuri

Introduction

The task at hand is to develop a program that solves Sudoku puzzles using Constraint Satisfaction Problem (CSP) techniques. Sudoku is a popular puzzle game where players fill a 9x9 grid with digits from 1 to 9, ensuring that each row, each column, and each of the nine 3x3 subgrids contain all of the digits from 1 to 9. The algorithm used in this task is backtracking with forward checking, a common approach to solving CSPs.

Backtracking is a general algorithmic technique for finding solutions to constraint satisfaction problems. It explores all possible candidates for the solution and eliminates those which cannot fulfill the constraints. Forward checking, on the other hand, is an enhancement to backtracking that reduces the search space by propagating information about variable assignments. This technique helps in detecting potential conflicts early, which can lead to more efficient pruning of the search tree.Implementation - Using snippets of code, explain step-by-step how the solution is working - Add images and/or visualizations requested by the task and explain them

Implementation

The solution is implemented in Python using a single class named CSP. Let's break down the implementation step-by-step:

1) Initialization: The puzzle grid, variables, domains, and assignment are initialized in the constructor (__init__ method). Variables represent the empty cells in the puzzle, domains hold possible values for each variable, and assignment keeps track of assigned values. def __init__(self, puzzle):

```
def __init__(self, puzzle):
    self.puzzle = puzzle
    self.variables = [(i, j) for i in range(9) for j in range(9) if puzzle[i][j] == 0]
    self.domains = {var: set(range(1, 10)) for var in self.variables}
    self.assignment = {}
```

2) Print Sudoku Puzzle: This method prints the Sudoku puzzle grid to the console. It formats the grid with appropriate separators and prints the numbers or dots representing empty cells.

```
def print_sudoku(self):
    for i in range(9):
      if i % 3 == 0 and i != 0:
        print("-" * 21)
      for j in range(9):
        if j % 3 == 0 and j != 0:
          print("| ", end="")
        print(self.puzzle[i][j] if self.puzzle[i][j] != 0 else ".", end=" ")
      print()
```

3) Visualize Sudoku Puzzle: This method creates a graphical representation of the Sudoku puzzle using matplotlib. It visualizes the puzzle as a grid with numbers in their respective cells. If final is True, it shows the final solved Sudoku puzzle.

```
def visualize(self, final=False):

    fig, ax = plt.subplots(figsize=(5, 5))
    ax.matshow(np.ones((9, 9)), cmap="Wistia")

    for i in range(9):
      for j in range(9):
        c = self.puzzle[j][i]
        ax.text(i, j, str(c), va='center', ha='center')

    for i in range(1, 9):
      if i % 3 == 0:
        lw = 2
      else:
        lw = 0.5
      ax.axhline(i-0.5, color='black', linewidth=lw)
      ax.axvline(i-0.5, color='black', linewidth=lw)

    plt.axis('off')
    if final:
      plt.show()
```

4) Check Value Is Consistent: This method checks if assigning a value to a variable is consistent with the constraints of the Sudoku puzzle. It verifies whether the assigned value conflicts with any existing values in the same row, column, or 3x3 subgrid.

```
def is_consistent(self, var, value):
    i, j = var
    for x in range(9):
        if self.puzzle[x][j] == value or self.puzzle[i][x] == value:
            return False
    box_start_i, box_start_j = 3 * (i // 3), 3 * (j // 3)
    for x in range(box_start_i, box_start_i + 3):
        for y in range(box_start_j, box_start_j + 3):
            if self.puzzle[x][y] == value:
                return False
    return True
```

5) Forward Checking: This method performs forward checking after assigning a value to a variable. It removes the assigned value from the domains of neighboring variables (in the same row, column, and 3x3 subgrid) to reduce the search space.

```
def forward_checking(self, var, value):
    removed_values = {}
    i, j = var
    for x in range(9):
        if (x, j) in self.variables and value in self.domains[(x, j)]:
            self.domains[(x, j)].remove(value)
            removed_values[(x, j)] = value
        if (i, x) in self.variables and value in self.domains[(i, x)]:
            self.domains[(i, x)].remove(value)
            removed_values[(i, x)] = value
    box_start_i, box_start_j = 3 * (i // 3), 3 * (j // 3)
    for x in range(box_start_i, box_start_i + 3):
        for y in range(box_start_j, box_start_j + 3):
            if (x, y) in self.variables and value in self.domains[(x, y)]:
                self.domains[(x, y)].remove(value)
                removed_values[(x, y)] = value
    return removed_values
```

6) Undo Forward Checking: This method reverses the effects of forward checking by adding back the removed values to the domains of the affected variables.

```
def undo_forward_checking(self, removed_values):
    for var, value in removed_values.items():
        self.domains[var].add(value)
```

7) Select Unassigned Variable: This method selects an unassigned variable from the puzzle. It returns the variable with the fewest remaining values in its domain, using a minimum remaining values heuristic.

```
def select_unassigned_variable(self):

    return min([v for v in self.variables if v not in self.assignment], key=lambda
    var: len(self.domains[var]), default=None)
```

8) Order Domain Values: This method orders the domain values of a variable. It returns the values in the domain of the given variable in their natural order.

```
def order_domain_values(self, var):
    if var is None:
        return []
    return list(self.domains[var])
```

9) Assign Values: This method assigns a value to a variable and updates the puzzle grid accordingly.

```
def assign(self, var, value):
    self.assignment[var] = value
    self.puzzle[var[0]][var[1]] = value
```

10) Unassign Values: This method unassigns a value from a variable and updates the puzzle grid accordingly.

```
def unassign(self, var):

    if var in self.assignment:

        del self.assignment[var]

        self.puzzle[var[0]][var[1]] = 0
```

11) Backtracking Search: The backtrack method recursively explores possible assignments to variables until a solution is found or all variables are assigned. It selects an unassigned variable, orders its domain values, and checks consistency before assigning a value. If inconsistency is detected, it backtracks and tries another value.

```
def backtrack(self):
    if len(self.assignment) == len(self.variables):
        return True

    var = self.select_unassigned_variable()

    for value in self.order_domain_values(var):
        if self.is_consistent(var, value):
            self.assign(var, value)
            removed_values = self.forward_checking(var, value)
            result = self.backtrack()
            if result:
                return result
            self.unassign(var)
            self.undo_forward_checking(removed_values)

    return False
```

12) Solving the Puzzle: The solve method initiates the backtracking search process and returns the solved Sudoku puzzle if a solution is found, otherwise returns None.

```
def solve(self):
    if self.backtrack():
        return self.puzzle
    return None
```

Discussion

In this implementation, the algorithm uses backtracking with forward checking to efficiently explore the solution space of Sudoku puzzles. The use of forward checking helps in reducing the search space by eliminating inconsistent assignments early in the process.

Test Cases:

The code provides three example Sudoku puzzles (easy_puzzle, hard_puzzle, and unsolvable_puzzle) and tests the CSP solver on each puzzle, printing the original and solved Sudoku grids.

Here are provided examples of diverse Sudoku puzzles alongside their respective solutions.

```
Original Sudoku:
5 3 . | . 7 . | . . .
6 . . | 1 9 5 | . . .
. 9 8 | . . . | . 6 .
- - - - - - - - - - - -
8 . . | . 6 . | . . 3
4 . . | 8 . 3 | . . 1
7 . . | . 2 . | . . 6
- - - - - - - - - - - -
. 6 . | . . . | 2 8 .
. . . | 4 1 9 | . . 5
. . . | . 8 . | . 7 9

Solved Sudoku:
5 3 4 | 6 7 8 | 9 1 2
6 7 2 | 1 9 5 | 3 4 8
1 9 8 | 3 4 2 | 5 6 7
- - - - - - - - - - - -
8 5 9 | 7 6 1 | 4 2 3
4 2 6 | 8 5 3 | 7 9 1
7 1 3 | 9 2 4 | 8 5 6
- - - - - - - - - - - -
9 6 1 | 5 3 7 | 2 8 4
2 8 7 | 4 1 9 | 6 3 5
3 4 5 | 2 8 6 | 1 7 9
```

```
Testing puzzle 2
Original Sudoku:
8 . . | . . . | . . .
. . 3 | 6 . . | . . .
. 7 . | . 9 . | 2 . .
--------------------
. 5 . | . . 7 | . . .
. . . | . 4 5 | 7 . .
. . . | 1 . . | . 3 .
--------------------
. . 1 | . . . | . 6 8
. . 8 | 5 . . | . 1 .
. 9 . | . . . | 4 . .

Solved Sudoku:
8 1 2 | 7 5 3 | 6 4 9
9 4 3 | 6 8 2 | 1 7 5
6 7 5 | 4 9 1 | 2 8 3
--------------------
1 5 4 | 2 3 7 | 8 9 6
3 6 9 | 8 4 5 | 7 2 1
2 8 7 | 1 6 9 | 5 3 4
--------------------
5 2 1 | 9 7 4 | 3 6 8
4 3 8 | 5 2 6 | 9 1 7
7 9 6 | 3 1 8 | 4 5 2
```

5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9	2	4	8	5	6
9	6	1	5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

8	1	2	7	5	3	6	4	9
9	4	3	6	8	2	1	7	5
6	7	5	4	9	1	2	8	3
1	5	4	2	3	7	8	9	6
3	6	9	8	4	5	7	2	1
2	8	7	1	6	9	5	3	4
5	2	1	9	7	4	3	6	8
4	3	8	5	2	6	9	1	7
7	9	6	3	1	8	4	5	2

This serves as an illustration of an unsolvable Sudoku puzzle.

```
Testing puzzle 3
Original Sudoku:
5 1 6 | 8 4 9 | 7 3 2
3 . 7 | 6 . 5 | . . .
8 . 9 | 7 . . | . 6 5
---------------------
1 3 5 | . 6 . | 9 . 7
4 7 2 | 5 9 1 | . . 6
9 6 8 | 3 7 . | . 5 .
---------------------
2 5 3 | 1 8 6 | . 7 4
6 8 4 | 2 . 7 | 5 . .
7 9 1 | . 3 4 | 6 . .
No solution found.
```

Conclusion

In conclusion, this report has presented a Sudoku solver implemented using Constraint Satisfaction Problem (CSP) techniques. Through the implementation, we have learned about the backtracking algorithm with forward checking and its application in solving Sudoku puzzles. While the implementation demonstrates an effective approach to

solving Sudoku puzzles, further optimizations could be explored to improve solving efficiency, such as more sophisticated heuristic techniques for variable and value selection. Overall, this project has provided valuable insights into problem-solving using CSP techniques.